



MEMORANDUM

To: Professor Brendon Anderson, Mechanical Engineering, California Polytechnic State University, bga@calpoly.edu
From: Gabriel Choi, Carmen Alaichamy
Date: May 6, 2026
Subject: Lab 3B: Closed-loop Velocity Control

Introduction

The objective of this lab was programming a PI controller using object-oriented programming and implement it on a microcontroller. This allowed us to test and evaluate the controller, testing the performance of the P-only, I-only, and PI controllers using step-response tests.

Methodology

The experiment was conducted using an STM32 microcontroller connected to a DC motor and rotary encoder. We completed the class file controller.py shown in Appendix A to control velocity of the motor using the encoder information. We created a constructor `__init__()` to set the PI gains. Then methods `set_setpoint()` and `set_gains()` sets the setpoint and controller gains. Finally method `run()` takes in the current velocity, and returns the correct voltage the motor should run at. The class file was then uploaded to the microcontroller using Thonny.

A main program (main.py) attached in Appendix B was developed to perform a step response test. This file performs the closed-loop step response test.

The collected data was plotted using Python in a Jupyter Notebook, documented in Appendix C, showing the motor's step response and controller output with corresponding K_p and K_I values. This allowed us to compare the effects of the proportional controller, integral controller, and both applied at the same time.

A Simulink model was created using the open-loop model, with the PI controller, DC model, and unity feedback. This was set to the optimal PI gains found in the last plot.

Hand calculations were also completed, calculating the theoretical time constant and steady-state values.

Results

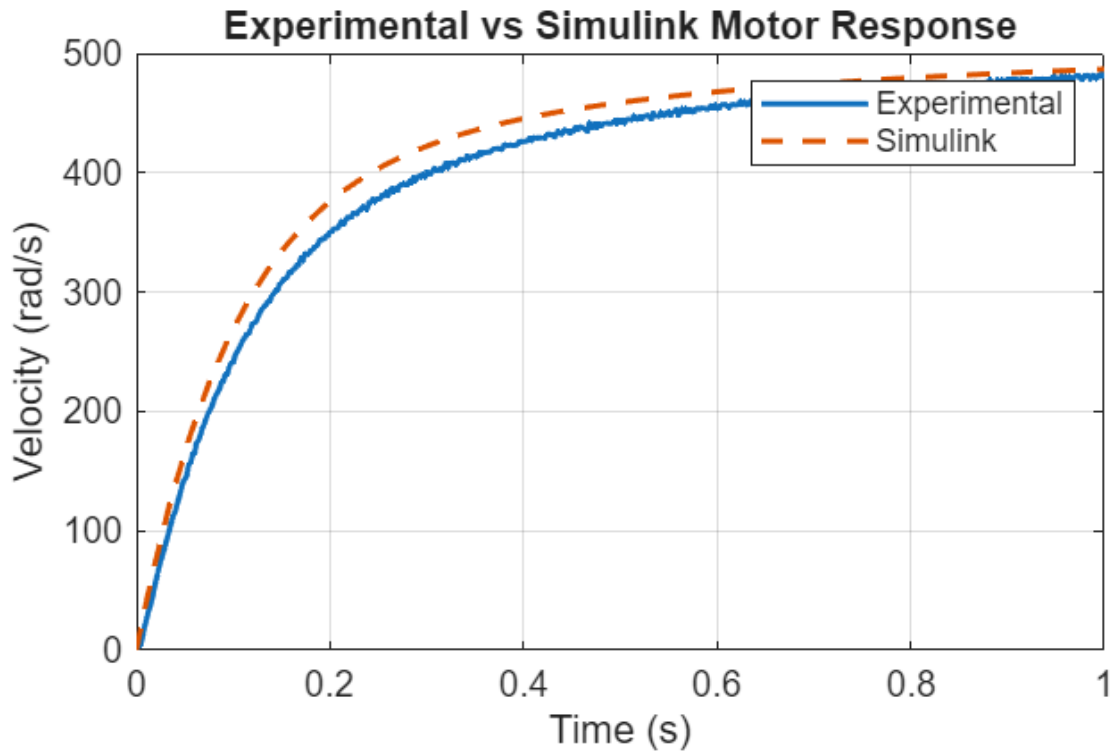


Figure 1. Optimal experimental and theoretical step response overlaid.

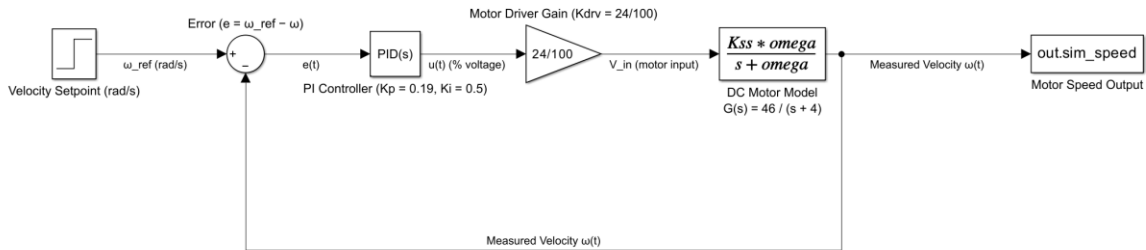


Figure 2. Simulink Motor

Comment on the differences between experimental and simulated step responses. What factors could contribute to the difference?

The simulated response generally follows the trend of the experimental motor speed response, but there are noticeable differences in both the transient and steady-state behavior. The experimental system rises faster and reaches steady-state more quickly, while the simulated response is slightly slower and smoother.

These differences arise because the simulation uses a simplified first-order model of the DC motor, which does not capture all real-world dynamics. In practice, the motor is affected by nonlinear effects such as friction, backlash, and saturation in the motor driver, which are not included in the model. Additionally, measurement noise from the encoder and delays in the microcontroller sampling can impact the experimental response. The PI controller implementation in hardware may also introduce discretization effects that are not present in the continuous-time Simulink model.

Overall, while the model captures the general behavior of the system, these unmodeled dynamics and real-world imperfections lead to the observed discrepancies between the experimental and simulated results.

Using:

$$K_p = 0.19$$

$$K_i = 0.5$$

$$K_{ss} = 46$$

$$\omega = 4$$

$$\text{Motor driver gain} = 24/100 = 0.24$$

Motor model:

$$G_m(s) = \frac{46(4)}{s + 4} = \frac{184}{s + 4}$$

Including motor driver:

$$G_p(s) = 0.24 \frac{184}{s + 4} = \frac{44.16}{s + 4}$$

PI controller:

$$G_c(s) = K_p + \frac{K_i}{s} = 0.19 + \frac{0.5}{s}$$

Open-loop transfer function:

$$G_{OL}(s) = \left(0.19 + \frac{0.5}{s}\right) \frac{44.16}{s + 4}$$

Closed-loop transfer function:

$$G_{CL}(s) = \frac{G_{OL}(s)}{1 + G_{OL}(s)}$$

$$G_{CL}(s) = \frac{8.3904s + 22.08}{s^2 + 12.3904s + 22.08}$$

The closed-loop poles are approximately:

$$s = -10.23, s = -2.16$$

Because both poles are real and negative, the theoretical system is stable and overdamped. This agrees with the experimental response because the motor speed rises smoothly without large oscillations.

Figure 2. Theoretical Calculations

The theoretical closed-loop model predicts a stable overdamped response with no oscillation, which matches the general shape of the experimental step response. The simulated response approaches the same final velocity as the experimental data, although small differences occur because the model is simplified and does not include friction, motor saturation, encoder noise, or sampling delays.

Discussion

Discuss effects of proportional and integral gains on transient and steady-state responses.

The proportional and integral controllers serve different functions in affecting the step response. By testing proportional and integral controllers individually, then combined, we can see their different effects. Proportional controller makes systems faster, but the steady state response will still have error. While the integral controller decreases error, it takes a long time to reach steady state so the main portion of the response is transient.

This is proven with the plots in Appendix C, where in Figure C.2 the step response from only the proportional controller, where it shows that the plot reaches steady state quickly but has large steady state error. With only integral controller, in Figure C.3, the response takes too long to reach steady state however it is closer to the correct value. This is because the integral works over time.

How would you graphically determine whether the controller saturates?

Graphically, controller saturation can be identified when the controller output reaches a maximum or minimum limit and flattens out, even though the velocity error is still changing. For this motor driver, saturation would appear as the control effort staying near $\pm 100\%$ voltage command.

In this experiment, why doesn't a large proportional gain alone produce oscillation, while a large integral gain does? Explain your answer using the location of closed loop poles with the PI controller and their corresponding time response.

In this experiment, a large proportional gain alone acts like a overdamped response, where K_p just moves towards the goal, however it often stops before reaching it, creating a steady state error. K_I creates oscillation as the integral logs error over time, so it continues to corrects itself even as it passes the setpoint. The accumulated error causes the overshoot.

```

1 # Defines a new class named PIController
2 class PIController:
3     ''' A class for a proportional-integral (PI) controller.
4
5     Attributes:
6         kp: Proportional gain
7         ki: Integral gain.
8         out_max: Maximum magnitude of output, for positive and negative.
9                 If this is exceeded, saturation will be applied.
10        delta_t: The period which the controller is run at.
11        sp: Controller setpoint.
12
13    '''
14    # Defines the class's constructor, which is run
15    # when object of this class is created
16    def __init__(self, kp, ki, out_max, delta_t, sp):
17        # 'self' is a placeholder for an object of this class.
18        # therefore, self.kp is a variable belonging to objects of the
19        # PIController class
20        # Set the PI gains. The other variables should be set the same way,
21        # using the self keyword to make them part of the controller object.
22        self.kp = kp
23        self.ki = ki
24        self.out_max = out_max
25        self.delta_t = delta_t
26        self.sp = sp
27        self._integral = 0
28
29        # Variables used to represent controller actions
30        self._i_action = 0
31        self._p_action = 0
32
33
34    def set_gains(self, kp, ki):
35        ''' Sets the proportional and integral controller gains.
36
37        Args:
38            kp: proportional gain (/(rad/s))
39            ki: integral ((rad/s)*s)
40        ...
41        self.kp = kp
42        self.ki = ki
43
44    # WRITE YOUR CODE HERE to define set_setpoint. Follow the same pattern as
45    # set_gains uses.
46
47    def set_setpoint(self, sp):
48        self.sp = sp
49
50    # WRITE YOUR CODE HERE to define the method run which will
51    # take in the velocity of the motor and return the percent
52    # voltage to set the motor to.
53    def run(self, x):
54        ''' Runs the PI controller once.
55
56        The time interval used for the integral calculation is set as delta_t.
57        Make sure to call this method at that period for the proper results.
58
59        Args:
60            x: The current value of the variable of the system which is being
61              controlled. Usually represented by the symbol c.
62
63        Returns:
64            The control command which will attempt to drive the system to the

```

```

65     setpoint. Usually represented by the symbol u.
66     ...
67     # Compute the error as the setpoint minus the current value x
68
69     error = self.sp - x
70     # Add error * delta_t to the integral error variable self._integral
71     self._integral += error * self.delta_t
72
73     # Calculate the integral and proportional actions (mind the sign).
74     self._p_action = self.kp * error
75     self._i_action = self.ki * self._integral
76
77     # Calculate the controller output (total action)
78
79     output = self._p_action + self._i_action
80     # Apply saturation to the controller output (out_max)
81
82     if output > self.out_max:
83         output = self.out_max
84     elif output < -self.out_max:
85         output = -self.out_max
86
87
88     # Return the total controller action
89     return output
90
91 def get_PI_components(self):
92     ''' Gets the current proportional and integral actions.
93
94     Keep in mind that these values do not saturate, but the total
95     controller output does.
96
97     Returns:
98         A tuple (p_action, i_action), where p_action is the component of
99         the output from the proportional term, and i_action is the
100        component of the output from the integral term.
101
102        ...
103        return self._p_action, self._i_action
104
105
106 # Controller uni test.
107 # The code below will only run when this file is executed as a standalone
108 # program. This is a good practice to ensure that the code works properly
109 # before using it in the main.py file.
110 #
111 # Run this file using the green run button to test that the
112 # PI controller calculations work properly.
113 if __name__ == '__main__':
114     con = PIController(0,0,100,1,5)
115     con.set_setpoint(5)
116     con.set_gains(1, 1)
117     print("Expected result:\n(5, 5)\n(5, 10)\n(5, 15)\n(5, 20)\n(5, 25)")
118     print("Actual result:")
119     for i in range(5):
120         con.run(0)
121         print(con.get_PI_components())
122
123     print("\nExpected result:\n(0, 25)\n(0, 25)\n(0, 25)\n(0, 25)\n(0, 25)")
124     print("Actual result:")
125     for i in range(5):
126         con.run(5)
127         print(con.get_PI_components())
128
129     con.set_gains(100,0)

```

```
130 | print("\nExpected result:\n100")
131 | print("Actual result:")
132 | print(con.run(0))
133 |
134 | print("\nExpected result:\n-100")
135 | print("Actual result:")
136 | print(con.run(10))
```

```

1  import cqueue
2  import utime
3  import pyb
4  import encoder
5  import motor
6  import controller
7
8  #####
9  # 1. PC -> MICROCONTROLLER COMMUNICATION (PARAMETER VALUES).
10 # This code reads values from the serial port for gains, velocity to step to,
11 # and total time to run the test for. No need to modify anything.
12 while True:
13     try:
14         kp = float(input("kp: [voltage percent per rad/s of error]: "))
15         ki = float(input("ki: [voltage percent per rad of integral]: "))
16         sp = float(input("Velocity setpoint to step to [rad/s]: "))
17         out_max = float(input("Maximum motor voltage [percent]: "))
18         t_total = float(input("Total test time [ms]: "))
19     except ValueError:
20         print("Invalid input")
21         continue
22     else:
23         break
24
25 #####
26 # 2. OBJECT/VARIABLE SETUP
27 # This code sets up motor driver and encoder objects (same as lab 1A).
28 driver = motor.MotorDriver()
29 driver.motorA.enable()
30
31 encoder = encoder.Encoder(4, pyb.Pin.cpu.B6, pyb.Pin.cpu.B7)
32 encoder.zero()
33
34 # This code creates arrays for time, velocity data, and control command data.
35 # Units should be microseconds, rad/s, and percent voltage
36 time_queue = cqueue.FloatQueue(1000)
37 vel_queue = cqueue.FloatQueue(1000)
38 percent_queue = cqueue.FloatQueue(1000)
39 p_queue = cqueue.FloatQueue(1000)
40 i_queue = cqueue.FloatQueue(1000)
41
42 # This sets up variables used to run the timed loop at 500Hz. Units are
43 # microseconds for all variables ending in _us
44 period_us = 2000
45 test_time_us = int(t_total * 1000)
46 start_time_us = utime.ticks_us()
47 next_time_us = start_time_us
48 delta_t = period_us / 1e6 # convert to seconds
49
50 # This determines how often to collect data. The desired number of data
51 # points for a graph is about 500-1000. The variable 'every' controls how
52 # many controller runs happen for each data point, for example, data is
53 # collected every 5 runs, 10 runs, etc. This is necessary because
54 # Collecting too much data can cause the microcontroller to run out of memory.
55 every = test_time_us // (period_us * 500)
56 every = 1 if every == 0 else every
57 n_runs = 0
58
59 #####
60 # WRITE YOUR CODE HERE to create the PI controller object. Use the parameters
61 # collected from the serial port in the above section.
62 con = controller.PIController(0, 0, 100, delta_t, 0)
63 # Set the controller gains (may need to change object name)
64 con.set_gains(kp, ki)

```



```

65
66 #####
67 # 3. CLOSED LOOP STEP RESPONSE
68 # WRITE YOUR CODE HERE to set the velocity setpoint to the value read from the
69 # serial port.
70 con.set_setpoint(sp)
71
72
73 # The next 3 lines are responsible for running the timed loop.
74 while utime.ticks_diff(utime.ticks_us(), start_time_us) <= test_time_us:
75     if utime.ticks_diff(utime.ticks_us(), next_time_us) >= 0:
76         next_time_us = utime.ticks_add(next_time_us, period_us)
77         # Any code written here will be run at 500Hz maximum.
78         # Read the current encoder velocity in rad/s.
79         vel = encoder.get_velocity_rad()
80
81         # WRITE YOUR CODE HERE to to run the controller, storing the percent
82         # voltage in a variable, so it can be added to the data later.
83         percent = con.run(vel)
84
85         # WRITE YOUR CODE HERE to set the voltage percent on the motor.
86         driver.motorA.set_voltage_percent(percent)
87
88         # This if-statement checks if it is time to collect data.
89         if n_runs % every == 0:
90             # WRITE YOUR CODE HERE to gather the controller actions from the
91             # get_PI_components method of the PIController.
92             _p_action, _i_action = con.get_PI_components()
93
94             # Store the time for this run in the time array.
95             time_queue.put(delta_t * n_runs)
96             # WRITE YOUR CODE HERE to append the values to the other arrays.
97             vel_queue.put(vel)
98             percent_queue.put(percent)
99             p_queue.put(_p_action)
100            i_queue.put(_i_action)
101
102            # Increment the counter for number of runs.
103            n_runs += 1
104
105 # Stop the motor.
106 driver.motorA.set_voltage_percent(0)
107 #####
108
109 # 3. MICROCONTROLLER -> PC COMMUNICATION (TEST DATA).
110 # This code prints the data to the computer.
111 while time_queue.any():
112     print(time_queue.get(), vel_queue.get(), percent_queue.get(),
113           p_queue.get(), i_queue.get(), sep=',')
114     utime.sleep_ms(1) # constant printing can cause errors in Thonny
115 print('END')

```

APPENDIX C: Jupyter Notebook Code and Graphs

This cell will import all of the necessary libraries for this lab. These include the pyserial library which is used to communicate with the microcontroller, NumPy which is used to load data from a CSV file, and pyplot from matplotlib which is a plotting package with almost identical commands to MATLAB. Run this cell first.

```
[314]: from serial import Serial
import time
import numpy as np
from matplotlib import pyplot as plt
```

This cell defines a function which will plot the step response. No need to modify this cell.

```
[316]: def run_step(kp, ki, step_vel, t_step, filename, out_max=100):
    # On this line, give the name of the serial port which the Nucleo
    # is connected to, as a string. This should be something like
    # 'COMx' for Windows and
    # '/dev/tty.usbmodemXXXX' for Mac.
    # See the bottom right corner of Thonny for the correct COM port to use.
    serport = 'COM8'

    # Open the serial port, and give it the name ser.
    with Serial(serport, baudrate=115200, timeout=5) as ser:
        # Restart the microcontroller
        ser.write(b'\x02\x04')
        # Write the parameters to the microcontroller
        ser.write((str(kp) + '\r\n').encode())
        ser.write((str(ki) + '\r\n').encode())
        ser.write((str(step_vel) + '\r\n').encode())
        ser.write((str(out_max) + '\r\n').encode())
        ser.write((str(t_step) + '\r\n').encode())

    # Open the data file for writing, and give it the name datafile
    with open(filename, 'w') as datafile:
        while True:
            # Read the current line and decode from bytes to a string
            line = ser.readline().decode()
            # Print the line so we can see what the microcontroller is doing
            # print(line, end='')
            # If the line starts with END, then we know the data is complete
            if line[:3] == 'END':
                break
            # If the line has a comma, it is a line of CSV data.
            elif ',' in line:
                datafile.write(line.strip() + '\n')

    plot_step(kp, ki, step_vel, filename)
```

Make several cells here to call `run_step` for each of the required gains from the lab manual. Use a unique filename for the data files. An example is below which runs a step response with $K_P = 0.1$ [%/(rad/s)]. $K_I = 0$, the velocity to step to of 500 rad/s, total test time of 1000 ms (1 second), and filename to save data to of "data1.csv".

Add more cells below using the Cell menu, or by pressing the 'b' key while in cell mode (press escape to enter cell mode) and highlighting the bottom cell. Then add one call of `run_step` per cell for the different gains specified in the lab manual. Make the test time long enough to see the systems get to steady state.

This cell will import all of the necessary libraries for this lab. These include the pyserial library which is used to communicate with the microcontroller, NumPy which is used to load data from a CSV file, and pyplot from matplotlib which is a plotting package with almost identical commands to MATLAB. Run this cell first.

```
[314]: from serial import Serial
import time
import numpy as np
from matplotlib import pyplot as plt
```

This cell defines a function which will plot the step response. No need to modify this cell.

```
[315]: def plot_step(kp, ki, step_vel, filename):
    """ Plot the step response

    Gives a subplot for the velocity vs time, and for the controller outputs
    vs time. Also places the gains on the plot title.

    Args:
        kp:      Proportional Gain
        ki:      Integral Gain
        step_vel: Velocity to step to
    """
    # Import the data from the file
    data = np.genfromtxt(filename, delimiter=',')
    times = data[:,0]
    velocities = data[:,1]
    percents = data[:,2]
    p_percents = data[:,3]
    i_percents = data[:,4]

    # Make a figure and set the size to 6x6 in
    fig = plt.figure()
    plt.rcParams['figure.figsize'] = (6, 6)

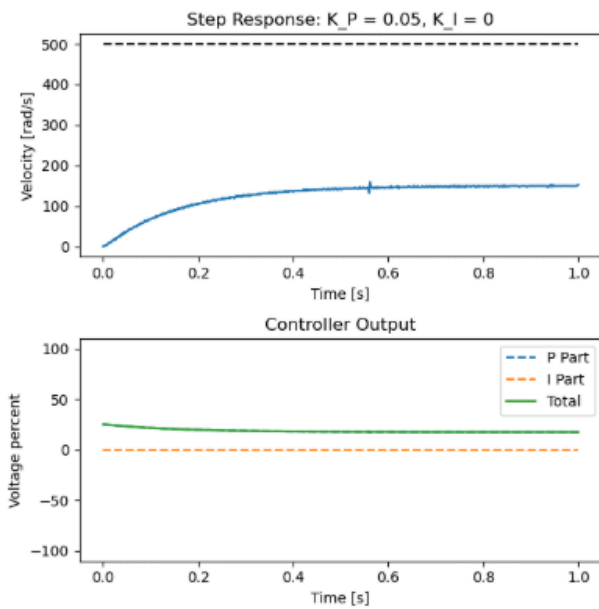
    # Plot the step response of position vs time on the top half
    plt.subplot(211)
    plt.plot(times, velocities)
    plt.hlines(step_vel, times[1], times[-1], colors='k', linestyle='dashed')
    plt.xlabel("Time [s]")
    plt.ylabel("Velocity [rad/s]")
    plt.title(f'Step Response: K_P = {kp}, K_I = {ki}')

    # Plot the proportional and integral components on the bottom half
    plt.subplot(212)
    plt.plot(times, p_percents, '--')
    plt.plot(times, i_percents, '--')
    plt.plot(times, percents)
    plt.ylim(-110, 110)
    plt.xlabel("Time [s]")
    plt.ylabel("Voltage percent")
    plt.title("Controller Output")
    plt.legend(['P Part', 'I Part', 'Total'])
    plt.tight_layout()
```

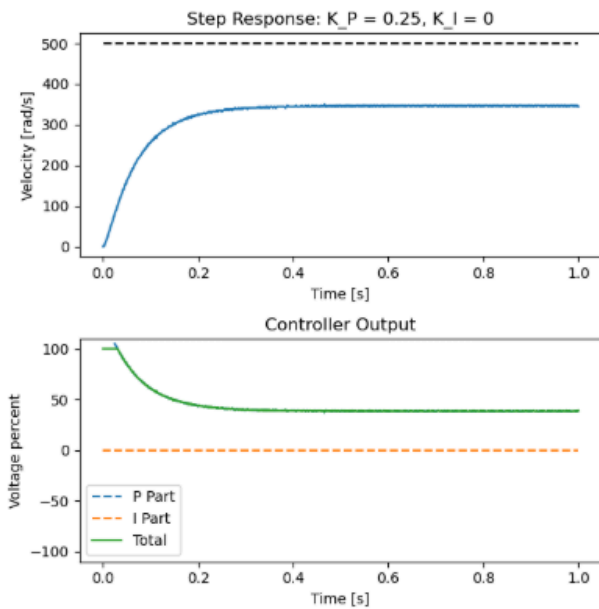
This cell defines a function which will connect to the microcontroller over the serial port. It will send the desired gains, velocity setpoint to step to, and total time to run the test for, as specified in the variables. Then it will collect the data and write it to `filename`. By default, the maximum value of the output is 100 (percent). Finally it will plot the step response using the previous function. No need to modify this cell, except to change the COM port.

Figure C.1. Jupyter Notebook code.

```
[317]: run_step(0.05, 0, 500, 1000, "kp_1.csv")
```



```
[318]: run_step(0.25, 0, 500, 1000, "kp_2.csv")
```



```
[319]: run_step(0.5, 0, 500, 1000, "kp_3.csv")
```

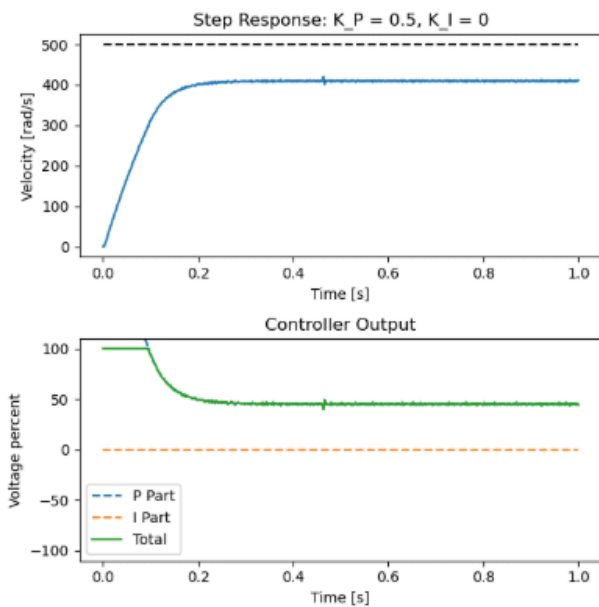
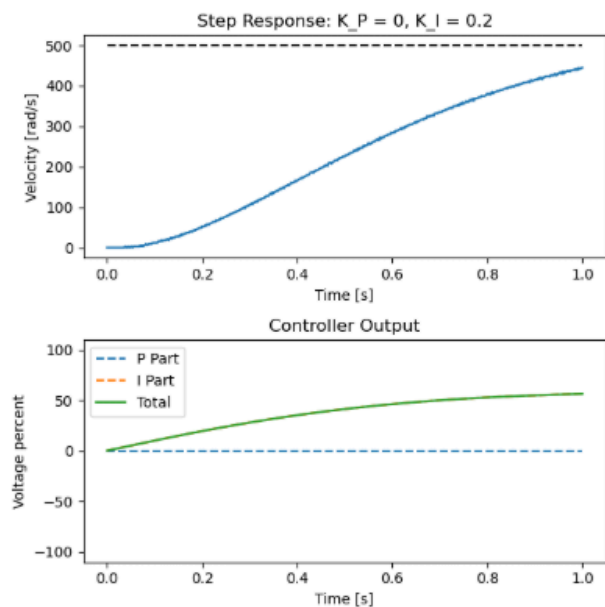
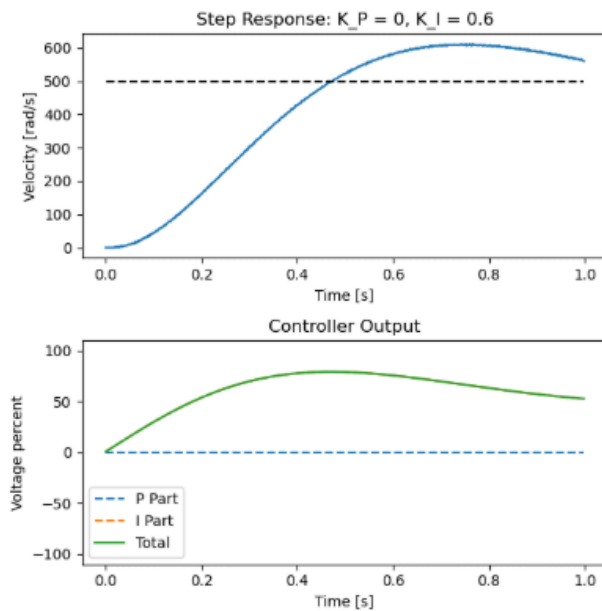


Figure C.2. Motor Step Response and Controller Output Graphs for $K_p = 0.05, 0.25$, and 0.5

```
[320]: run_step(0, 0.2, 500, 1000, "ki_1.csv")
```



```
[321]: run_step(0, 0.6, 500, 1000, "k1_2.csv")
```



```
[322]: run_step(0, 1, 500, 1000, "k1_3.csv")
```

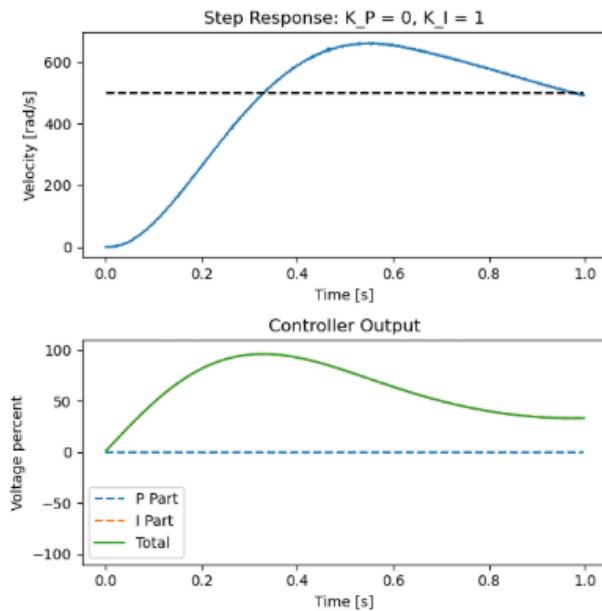


Figure C.3. Motor Step Response and Controller Output Graphs for $K_I = 0.2$, 0.6 , and 1 .

```
[323]: run_step(0.19, .5, 500, 1000, "kp_ki.csv")
```

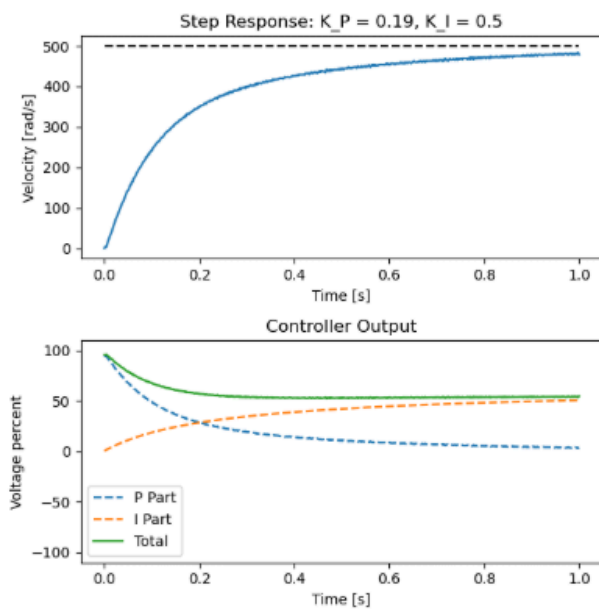


Figure C.4. Optimal Motor Step Response and Controller Output Graphs where $K_p = 0.19$ and $K_I = 0.5$.